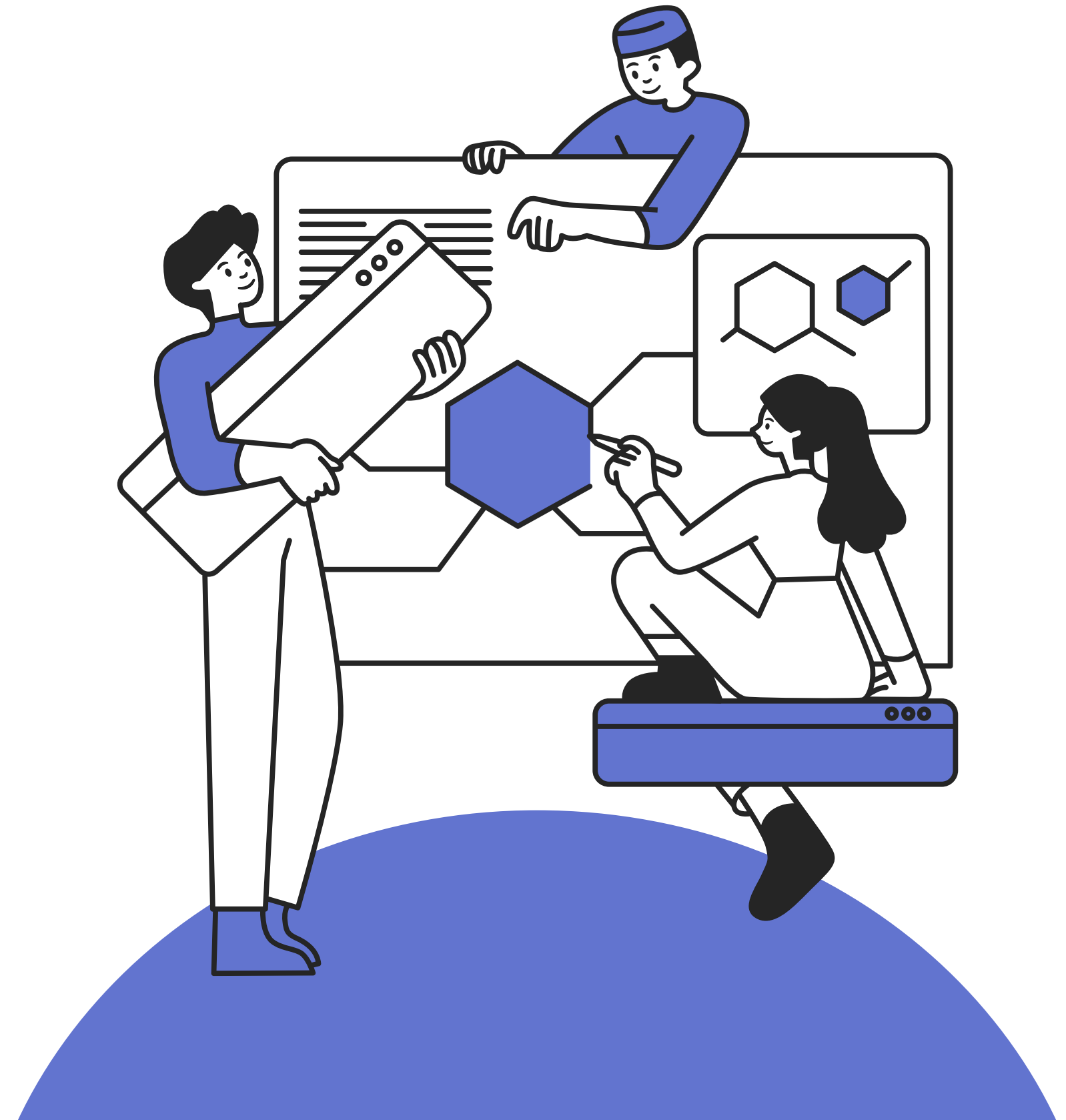


백트래킹

이예린T



낙화

가야 할 때가 언제인가를
분명히 알고 가는 이의
뒷모습은 얼마나 아름다운가

봄 한철
걱정을 인내한
나의 사랑은 지고 있다

분분한 낙화.....
결별이 이룩하는 축복에 싸여
자금은 가야 할 때.

무성한 녹음과 그리고
머지않아 열매 맺는 가을을 향하여
나의 청춘은 꽃답게 죽는다.

헤어지자
섬세한 손길을 흔들며
하롱하롱 꽃잎이 지는 어느 날

나의 사랑, 나의 결별,
샘터에 물 고이듯 성숙하는
내 영혼의 슬픈 눈



◆ AI 개요

'Backtrack'은 상황에 따라 다양한 의미로 사용됩니다. 주로 '시작점으로 되돌아가다', '이전 발언을 철회하다', '길을 되짚어 가다'의 뜻을 지니며, IT나 오디오 기록 플랫폼을 지칭할 때도 사용됩니다. [Cambridge Dictionary +2](#)

- **일상/비즈니스:** 앞서 했던 말이나 결정을 **철회**하거나 **반복**할 때 사용됩니다. (예: "The government backtracked on plans...") [Cambridge Dictionary](#)
- **일반 길찾기:** 왔던 길을 그대로 **되돌아가다**를 뜻합니다. [Cambridge Dictionary](#)
- **오디오/회의록 기록 앱:** 1:1 미팅이나 오디오 대화 내용을 최대 20분까지 역방향으로 녹음하고 AI가 요약해 주는 플랫폼 **[Backtrack]**을 의미합니다. usebacktrack.com
- **보안 운영체제(OS):** 과거 널리 사용되었던 정보 보안 테스트 및 해킹 점검용 리눅스 배포판으로, 현재는 후속 버전인 **[Kali Linux]**로 계승되었습니다. [BackTrack Linux +1](#)
- **컴퓨터 알고리즘:** 해를 찾는 과정에서 막히면 이전 상태로 되돌아가 다시 해를 찾는 기법인 **백트래킹(되추적)**을 뜻합니다. [Cambridge Dictionary +1](#)

3 학년 선택	생활· 교양	제2 외국어 교양	일반	일본어 I	5	4									
			일반	중국어 I	5	4					2(택1)	2(택1)		4	
				논술	5	4									
	전문	과학 계열	전문 I	고급 수학 I	5	4					16(택4)			36	
				AP미적분학 I	5	4									
				AP일반물리학 I	5	4									
				Conceptual 물리학 I	5	4									
				AP 일반 화학 I	5	4									
				유기화학 I	5	4									
				AP일반생물학	5	4									
				세포생물학	5	4									
				전문학및실험	5	4									
				정보과학	5	4									
				고급 수학Ⅱ	5	4					20(택5)				
				선형대수학	5	4									
				AP일반물리학Ⅱ	5	4									
				현대 물리학 I	5	4									
				AP 일반 화학Ⅱ	5	4									
				유기화학Ⅱ	5	4									
				생태와 환경	5	4									
				인간생활과 생명과학	5	4									
				지질학및실험	5	4									
				인공지능과 미래사회	5	4									
	생활·교양	기술·가정	진로												

빠른 벡 프래킹 도 기슬이라

- 00대학교 교수 한00 -

CONTENTS

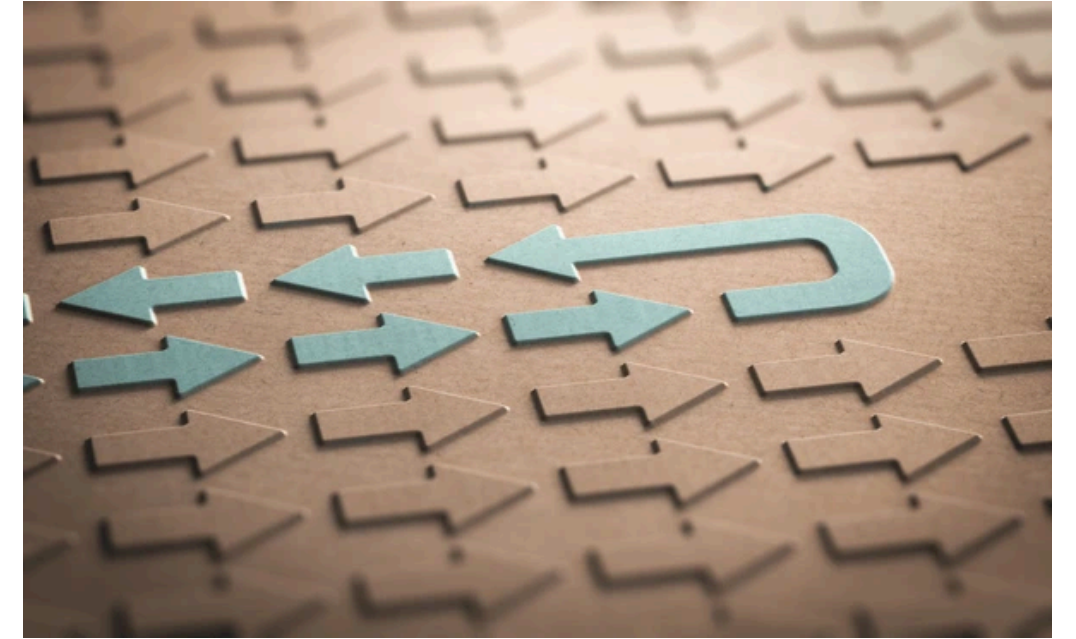
- 1 백트래킹
- 2 백트래킹 대표 문제
- 3 모듈별 탐구
- 4 백트래킹 정리

1 백트래킹

01

백트래킹

Backtracking



- 문제의 해가 될 수 있는 후보를 찾고, 해가 될 수 있는 조건을 충족하지 못하는 후보를 제거해 나가면서 최종 해를 찾는 기법
- 여러 후보해 중 특정 조건을 충족시키는 모든 해를 찾는 알고리즘
- 수많은 후보해 중 해가 될 조건을 만족시키는 ‘진짜 해’를 ‘효율적으로’ 찾는 것이 목적
- 기본 골격은 깊이우선탐색(DFS)이며, 단말 노드까지 가기 전에 방문한 노드(후보해)가 문제에서 요구하는 해가 될 수 없다고 판단되면 더는 탐색하지 않고 이전 노드로 되돌아 나오는 전략
- 상태공간트리의 많은 노드들을 탐색하지 않고 미리 가지치기(pruning)할 수 있어, 탐색의 효율을 높인다.
 - 상태공간트리(state space tree): 문제 해결 과정의 중간 상태들을 모두 노드로 구현해 놓은 트리

01

백트래킹의 설계

Backtracking

- 1) 해를 찾는 과정은 ‘뿌리’부터 출발한다.
- 2) 현재 위치한 부분해에서 선택 가능한 다음 부분해 목록을 얻는다.
- 3) 2)에서 얻은 목록의 부분해들을 하나씩 방문한다.
- 4) 방문한 부분해가 ‘해가 될 수 있는 조건’을 만족시키면 그 자리에서 단계 2)와 3)을 수행하고, 그렇지 않으면 그 이전 부분해로 돌아 나와 다른 부분해를 시도한다.
- 5) 최종해를 얻을 때까지 또는 모든 경우의 수를 확인해도 해가 없음을 확인할 때까지 2)~4)를 반복한다.

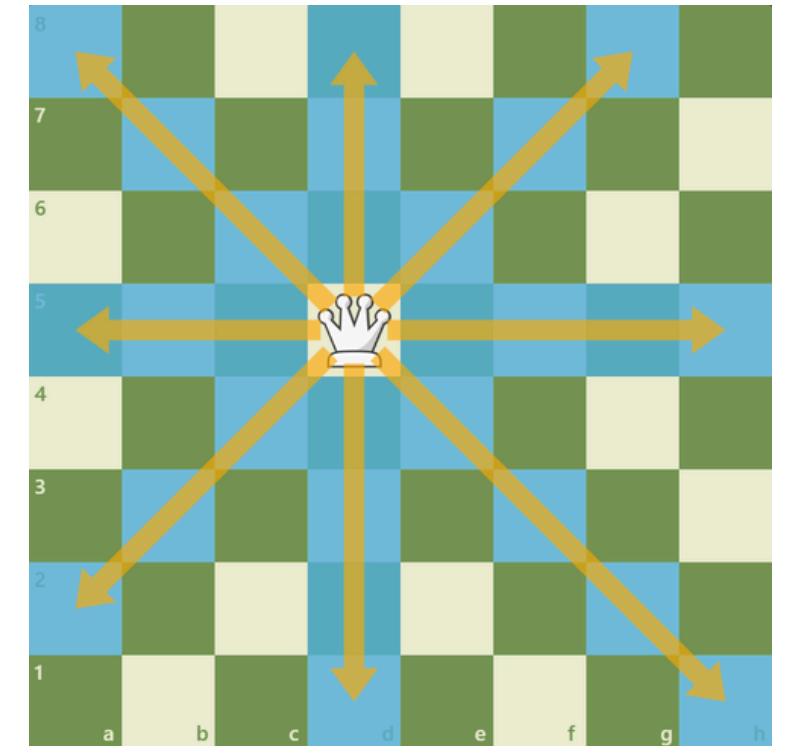
2 백트래킹 대표 문제

01

N-Queens

체스 퀸 배치하기

- <https://queenly-gameplay.lovable.app/>
- $N \times N$ 크기의 체스판 안에 N 개의 퀸을 서로 공격할 수 없도록 배치하는 모든 경우의 수 구하기
- 퀸: 수직, 수평, 대각선 방향으로 공격 가능
- 4×4 이상부터 적용 가능



01

N-Queens

체스 퀸 배치하기

- 모든 경우의 수를 계산하면 1820개 계산해야 하지만,
- 백트래킹을 이용하여 불필요한 후보해 검사량을 크게 줄임.

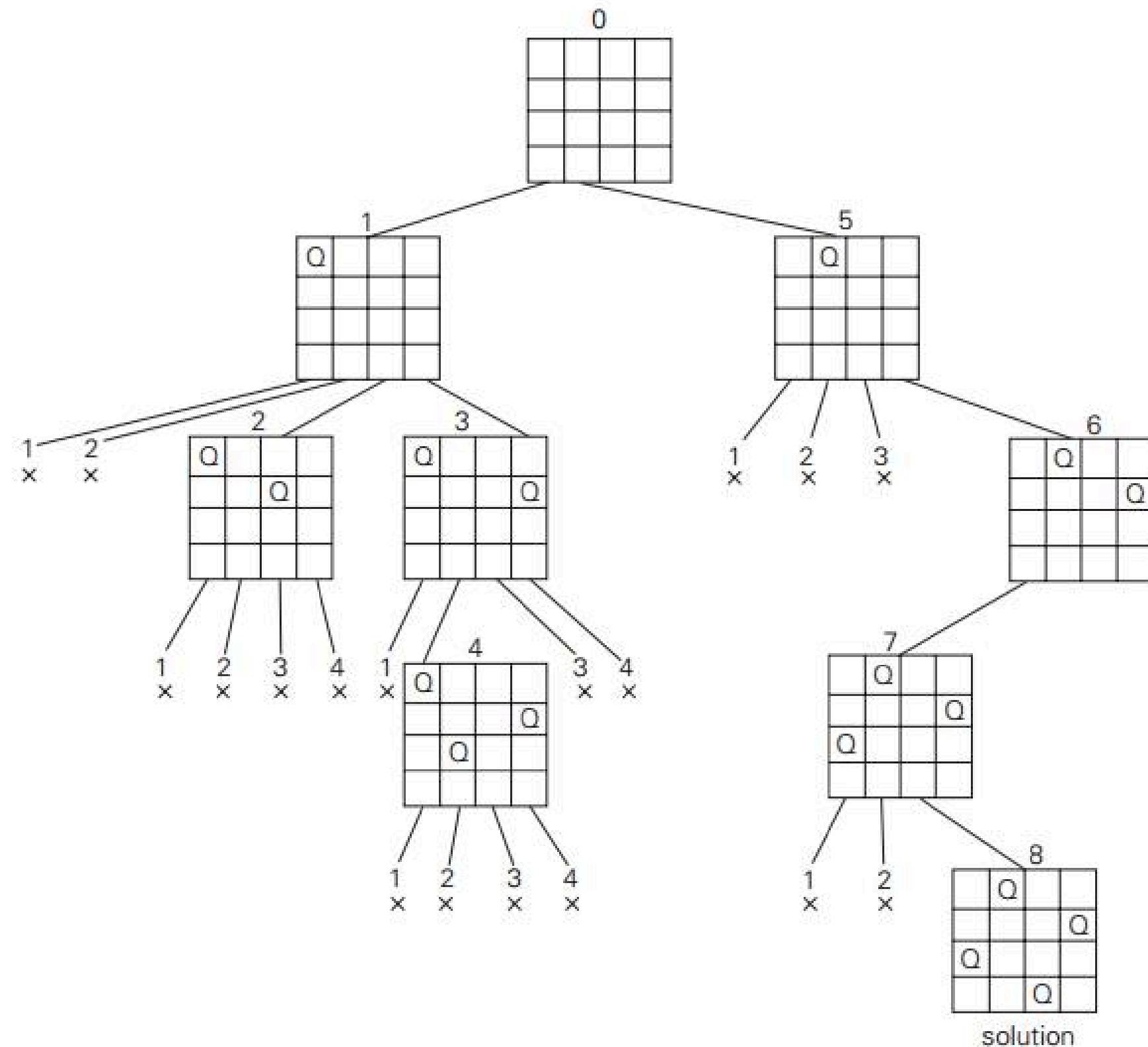


FIGURE 12.2 State-space tree of solving the four-queens problem by backtracking. x denotes an unsuccessful attempt to place a queen in the indicated column. The numbers above the nodes indicate the order in which the nodes are generated.

01

N-Queens

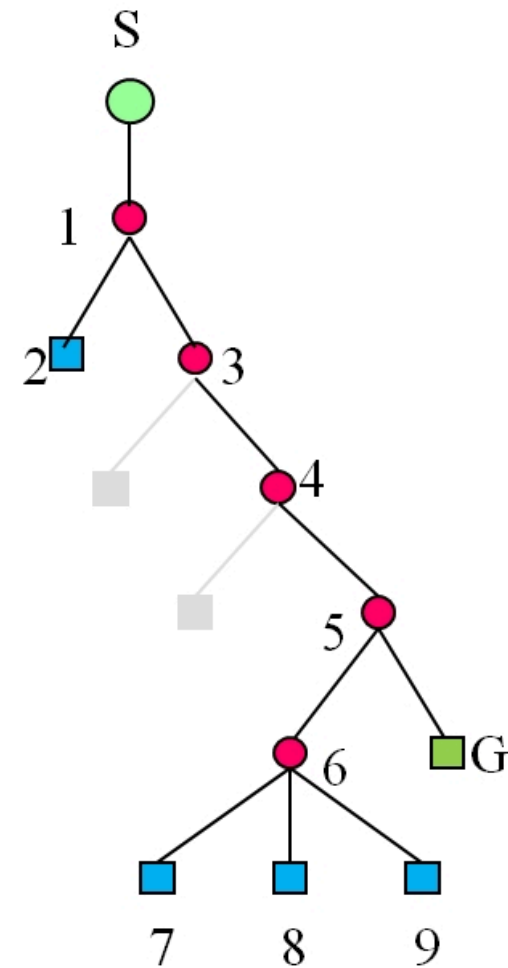
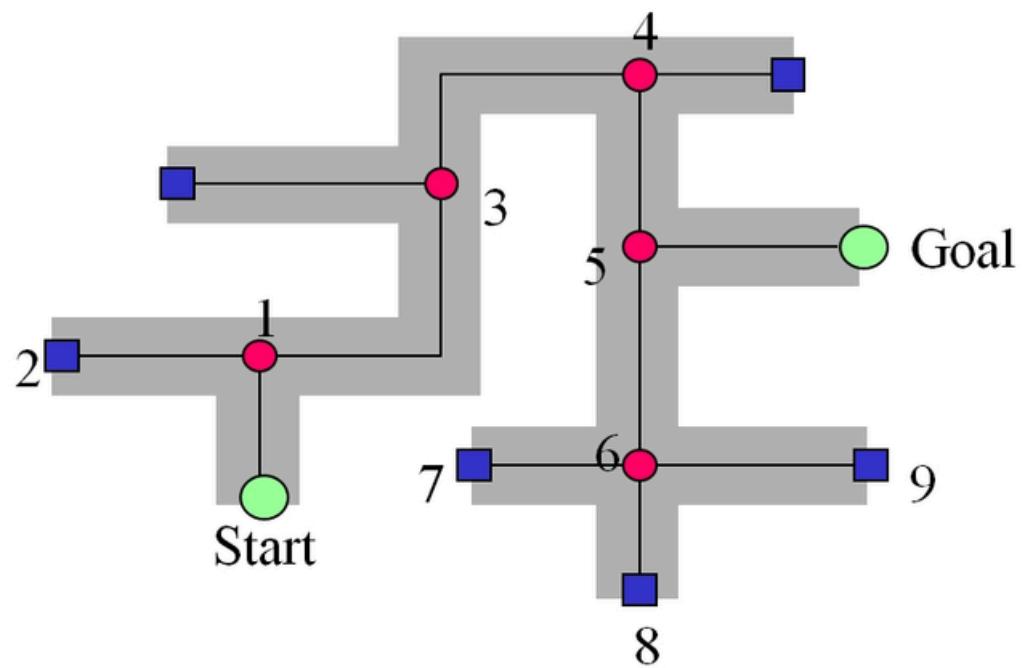
구현

- 해가 될 수 있는 조건: 수직, 수평, 대각선에 퀸이 없어야 함.
 - 대각선 방향 위협: 두 칸의 행 번호 차이 값과 열 번호 차이 값이 같으면 대각선 상에 놓인 것
- 자료구조: 1차원 배열 사용
 - 인덱스: 행 번호
 - 값: 열 번호

```
1 n = int(input())
2
3 ans = []
4 row = [0] * n
5
6 def is_promising(x):
7     for i in range(x):
8         if row[x] == row[i] or abs(row[x] - row[i]) == abs(x - i):
9             return False
10    return True
11
12 def n_queens(x):
13     if x == n:
14         ans.append(row[:]) # row[:] 로 복사해서 저장
15         return
16
17     else:
18         for i in range(n):
19             row[x] = i
20             if is_promising(x):
21                 n_queens(x + 1)
22
23 n_queens(0)
24
25 for sol in ans:
26     print(sol)
```

02

미로 탈출하기



- Start 노드
- 루트 노드
- 백트래킹 된 노드
- Goal 노드

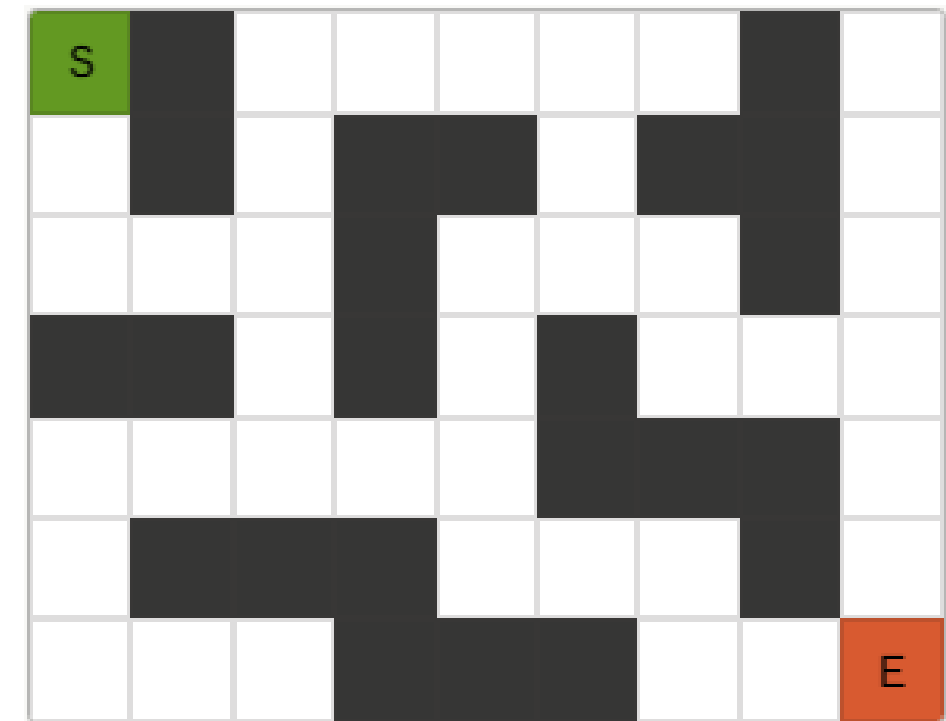
- 갈림길이 나오면 무조건 왼쪽 길부터 들어간다.
- 막다른 곳이 나오면 갈라진 길목으로 되돌아 나와 그다음 길을 시도한다.
- 목표 지점에 도달하거나 모든 경로를 다 탐색할 때까지 반복한다.
- 노드 이동(부분해 계산) → 자식 노드 목록 확인(다음 부분 후보해 목록 확인) → 각 자식 노드로 이동(각 부분 후보해 계산)

02

미로 탈출하기

구현

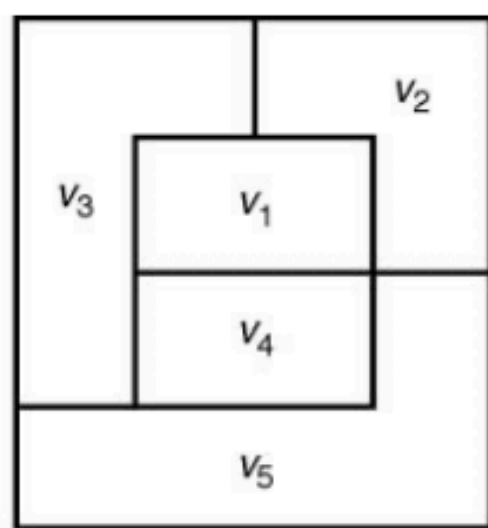
- 시작점 S를 현재 위치로 지정하고 이동 방향을 북으로 설정한다.
- 현 위치에서 가고자 하는 방향으로 이동 가능 여부를 확인한다.
 - 벽과 지나왔던 길은 이동 가능한 길이 아니다.
- 이동 가능하다면 그 방향으로 이동한다. 이동 불가능하면 방향(북-남-동-서 순서)을 바꿔 다시 이동 가능 여부를 확인한다. 현 위치에서 북, 남, 동, 서 어느 방향으로도 이동할 수 없음이 확인되면 이전 위치로 돌아간다.
- 출구를 찾거나 미로 내의 모든 길을 방문할 때까지 반복한다.



03

지도 색칠하기

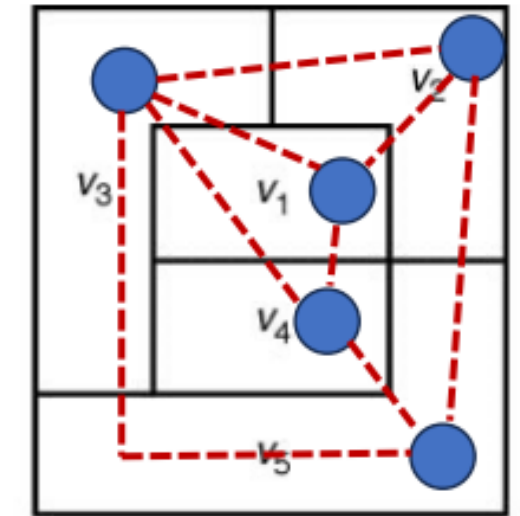
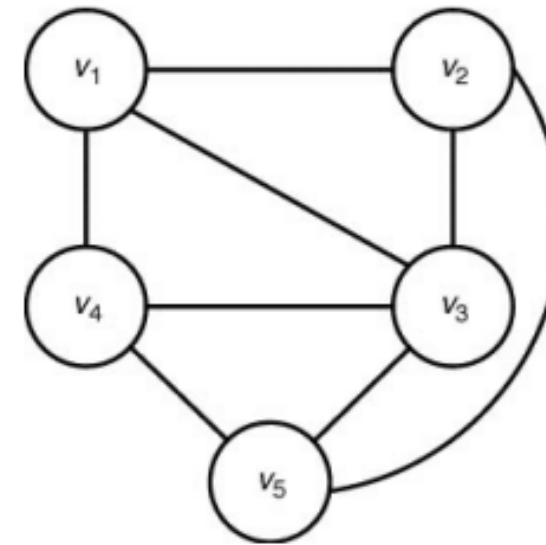
- 지도에서 인접하는 지역끼리는 다른 색으로 칠해야 영역을 구분할 수 있다.
- 인접한 지역에 같은 색이 할당되지 않도록 색칠하는 방법을 구하시오.
- 노드에 색을 배정할 때 인접 노드와 겹치면 → 다른 색 시도 → 모든 색이 안 되면 → 이전 노드로 백트래킹



03

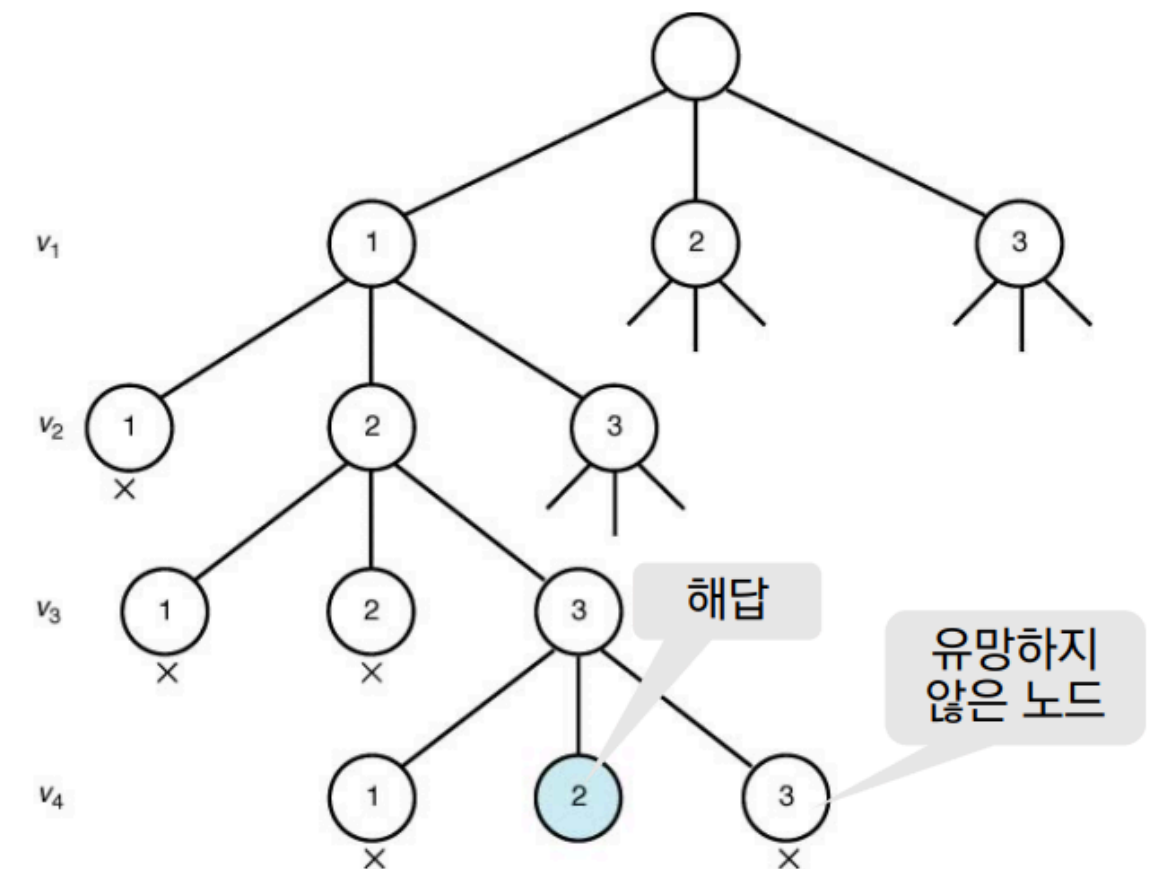
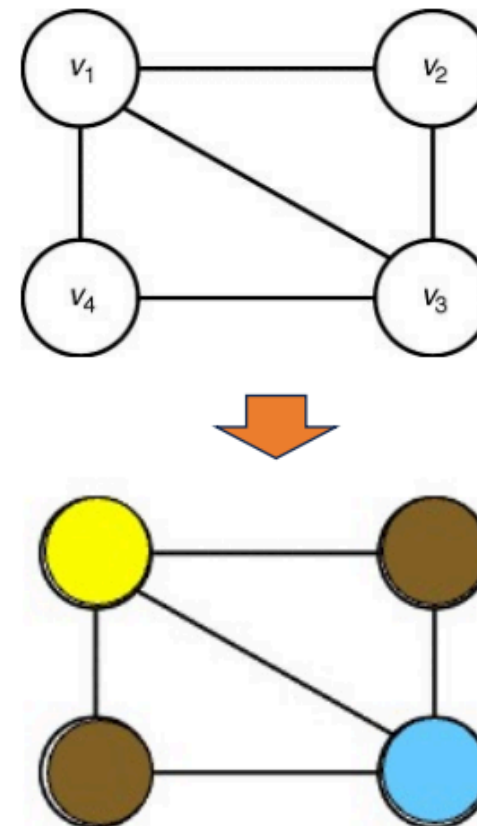
지도 색칠하기

- 지도의 각 지역을 그래프의 정점으로, 인접한 지역 사이에는 간선 연결



- 각 레벨은 노드 번호를 의미한다.
- 각 레벨에서 가능한 색상을 차례로 시도해 보며 마지막 노드까지 반복한다.
- 유망하지 않은 노드는 더이상 탐색하지 않는다.

3가지 색깔 사용할 경우의 해 찾기

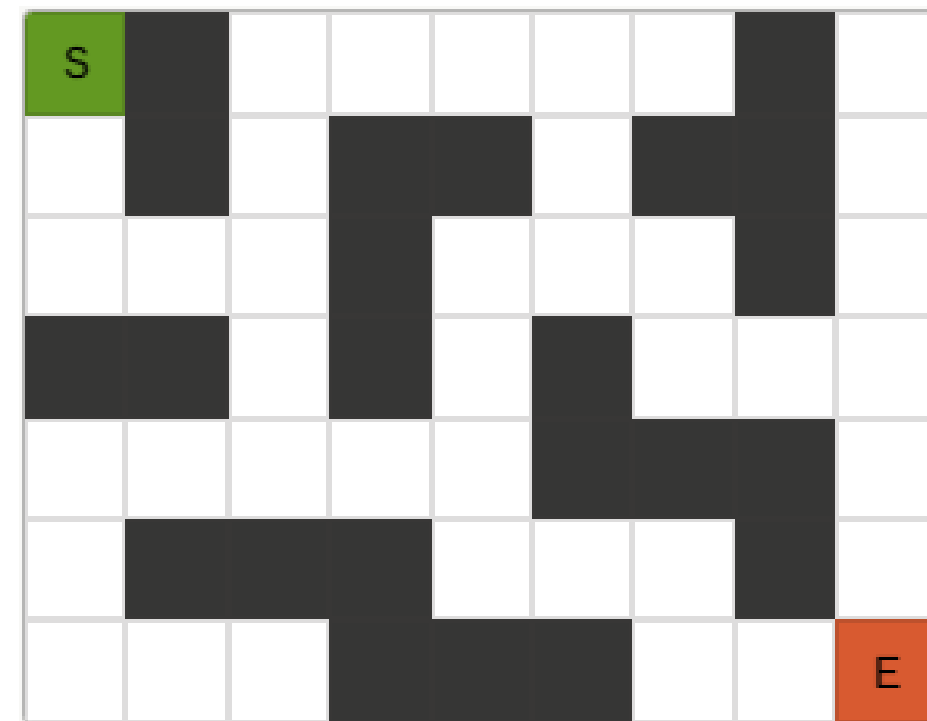
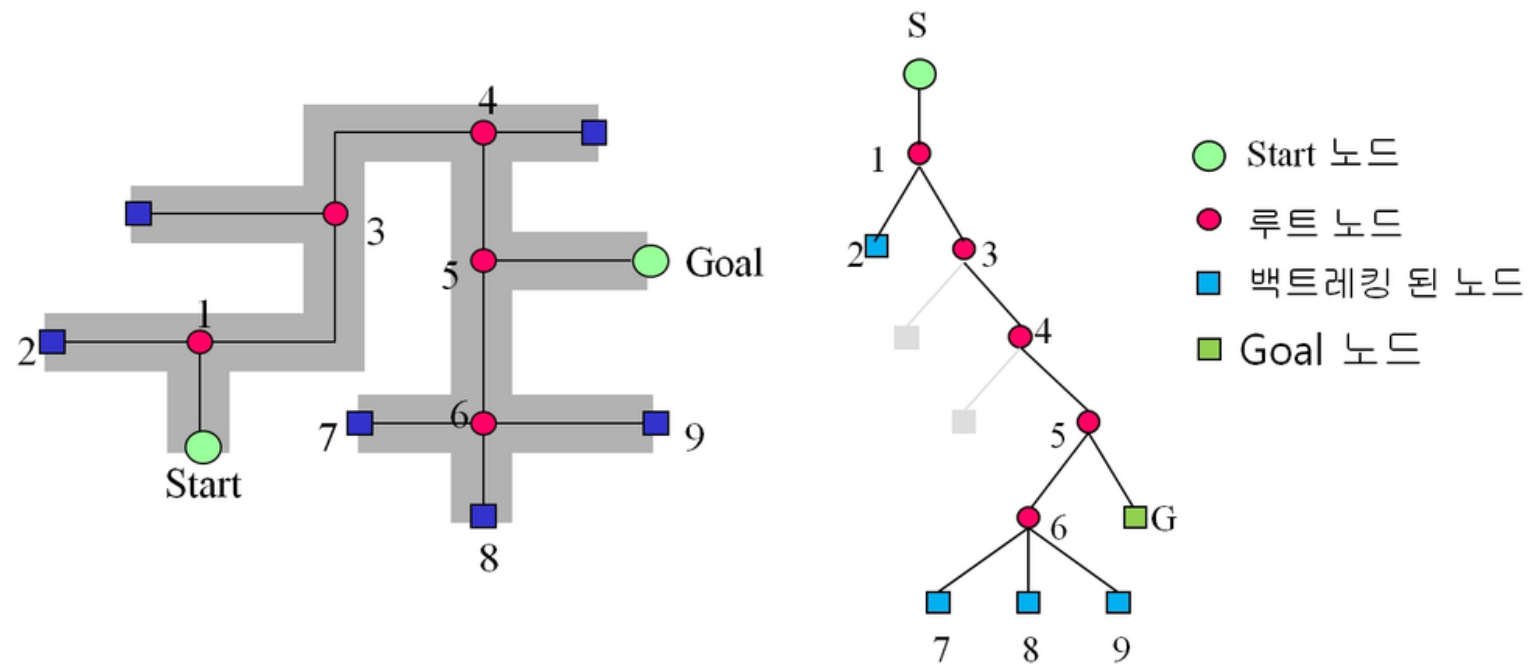


3 모듈별 탐구

01

미로 탈출하기 구현

- 미로를 입력받아 탈출 경로를 출력하시오.
 - 길은 0, 벽은 1로 입력받는다.
 - 시작은 (0,0), 탈출은 (-1,-1)
 - 탈출 경로 한 가지만 출력되면 됨.
 - 탈출이 불가능한 미로라면 탈출 불가능하다는 메시지를 출력한다.
 - 수업에서 다른 구현 방법을 따른다.
- 아래 미로와 트리를 참고하여, 오른쪽 미로를 트리로 나타내시오.
 - 노드 순서는 북-남-동-서 이동 방향에 따라 번호 붙인다.



02

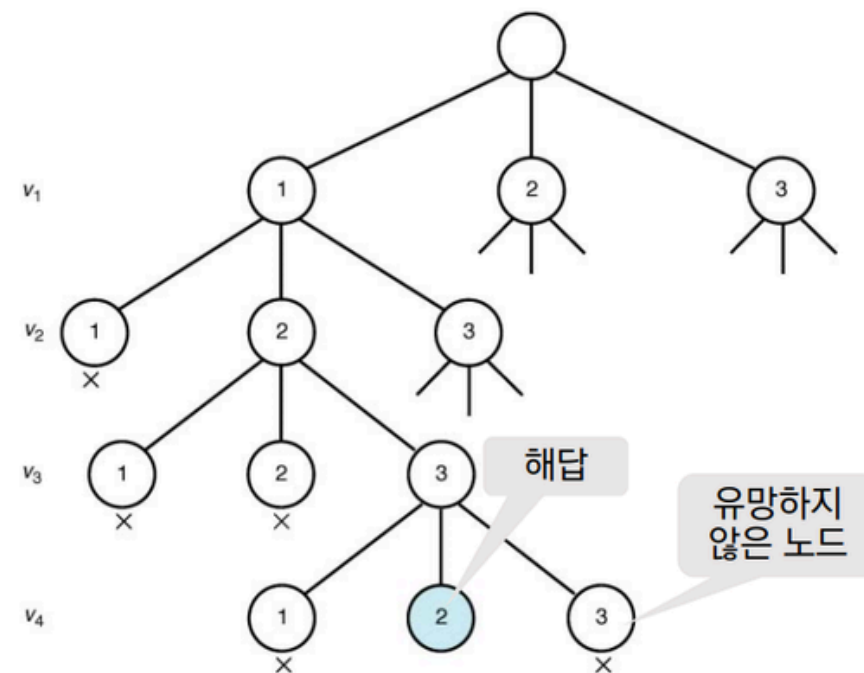
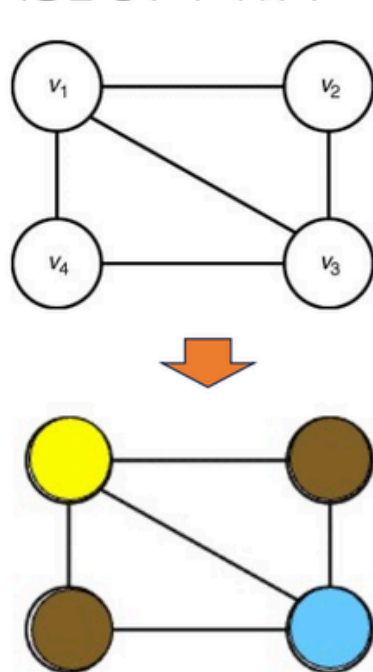
지도 색칠하기 구현

- 구로구 동작구 0
- 마포구 중구 X
- 종로구 성동구 0
- 관악구 구로구 0

- 동작구 마포구 X
- 서대문구 용산구 X

- 서울시 지도를 인접 리스트(자료형은 딕셔너리)로 구현하시오. (각 지역구가 노드가 되도록. 강변에 붙은 구끼리는 인접한 것으로 봄.)
- 몇 가지 색상을 사용할지 입력 받고, 노드별로 몇 번 색상으로 칠할지 출력되도록 하시오.
 - 서울시 지도 인접 리스트를 저장해 놓으시오.
 - 다른 지도를 입력해도 정상적으로 작동해야 함.
- 서울시 지도 색칠 방법을 아래와 같은 트리로 나타내시오.
- 10번째 노드(level 10) 이후로는 생략 처리(색상 4개)

3가지 색깔 사용할 경우의 해 찾기



03

순열 생성

- N 개의 서로 다른 숫자 $[1, 2, \dots, N]$ 이 주어질 때, 만들 수 있는 모든 순열을 출력하라.
 - N 을 입력 받고, 1부터 N 까지의 숫자를 사용해 N 길이의 순열을 생성함.
 - 백트래킹을 사용하여 구현할 것
 - 같은 숫자는 한 번만 사용할 것
- $1, 2, \dots, N$ 까지의 숫자 중, k 개의 숫자를 사용해 만들 수 있는 모든 순열을 출력하라.
 - N 과 k 를 입력 받는다.
 - 백트래킹을 사용하여 구현할 것
 - 같은 숫자는 한 번만 사용할 것

04

스도쿠 풀기

- 4×4 스도쿠 판의 빈 칸(0)을 채워라. 아래 세 가지 조건을 동시에 만족해야 한다.
- 각 행에 1~4가 한 번씩만 등장
- 각 열에 1~4가 한 번씩만 등장
- 각 2×2 박스에 1~4가 한 번씩만 등장
- 입력값으로는 스도쿠 규칙을 위반하지 않는 판만 입력된다.
 - 예)
 - 입력:
 - [0, 0, 0, 1]
 - [0, 1, 0, 0]
 - [0, 0, 2, 0]
 - [3, 0, 0, 0]
 - 출력:
 - [2, 3, 4, 1]
 - [4, 1, 3, 2]
 - [1, 4, 2, 3]
 - [3, 2, 1, 4]
- 조건
 - 백트래킹을 사용하여 구현할 것
 - is_safe(row, col, num) 함수를 별도로 작성할 것
 - "(row, col) 위치에 num을 놓아도 괜찮은가?" 를 검사하는 함수
 - 백트래킹이 일어나는 시점을 코드 상에 주석으로 표기할 것
- 다음을 서술하시오.
 - is_safe에서 체크하는 조건 3가지는 무엇인가?
 - N-Queens의 is_promising과 이 문제의 is_safe는 어떤 점이 같고 다른가?

05

숫자 골라 최대 합 만들기

- n 행 m 열 격자판의 각 칸에 정수가 들어있다. 이 격자판에서 칸 k 개를 선택하려고 한다. 선택한 칸들은 인접하면 안 되고, 기준 칸에서 위, 아래, 왼쪽, 오른쪽 칸이 인접한 칸이다. 선택한 칸에 들어있는 수들의 합의 최댓값을 구하시오.
 - 단, n 과 m 은 $[1, 10]$ 에 속한 정수고, k 는 $n \times m$ 과 4 중 최솟값 이하이다.
- 백트래킹이 일어나는 조건을 모두 서술하시오.
- 예)
 - 입력: (n, m, k , 격자판 정수들)
 - 3 3 2
 - 1 2 3
 - 4 5 6
 - 7 8 9
 - 출력 16
- 예)
 - 입력:
 - 2 3 3
 - 10 -1 10
 - -1 10 -1
 - 출력: 30

4 백트래킹 정리

01

왜 백트래킹을 쓰는가?

Backtracking

- **막다른 길이 나오면 분기점으로 돌아가 다른 길을 시도한다.**
- 해를 한 단계씩 구성해 나가다가, 더 이상 답이 될 수 없음이 확실해지는 순간 그 경로를 즉시 중단하고 갈라졌던 분기점으로 되돌아간다.
- 선택 → 탐색 → 실패 시 복원
- 최악의 경우 완전탐색과 방문 노드 수가 같을 수 있다.
- ‘답이 될 수 없음’이 확인되면 **가지치기**를 통해 다시는 그 길로 가지 않는다.
 - 가지치기를 통해 방문 노드 수를 줄인다.

01

왜 백트래킹을 쓰는가?

핵심은 가지치기

- **완전한 해를 끝까지 만들기 전에**, 지금까지 구성한 부분해가 제약조건을 위반하는지 먼저 검사한다. 위반사항이 확인되면 그 지점에서 더 깊이 탐색하지 않고 즉시 멈춘다.
- 미로 길목에서 한쪽 길을 선택했을 때 막다른 길이 나오면 갈라졌던 길목으로 되돌아 와서 다시는 그 길로 가지 않게 표시해 두는 것
- 백트래킹의 효율화 전략
- 최악의 시간복잡도(빅오)는 완전탐색과 같을 수 있지만, 가지치기를 통해 실제로 방문하는 노드 수는 크게 줄어든다. → 이론적 빅오는 나빠도, 실전 성능은 좋아진다.



02

스도쿠의 구현

- is_safe에서 체크하는 조건 3가지는 무엇인가?
 - 같은 행에 동일한 숫자가 있는가
 - 같은 열에 동일한 숫자가 있는가
 - 같은 3*3 박스에 동일한 숫자가 있는가
- N-Queens의 is_promising과 이 문제의 is_safe는 어떤 점이 같고 다른가?
 - 같은 점: 둘 다 가지치기 함수다.
 - 새로 놓는 값이 이미 배치된 값들과 제약을 위반하지 검사한다.
 - 검사 결과가 True일 때만 다음 깊이로 재귀 호출
 - 다른 점: 제약사항 개수(2개, 3개)

02

스도쿠의 구현

```

13 def is_safe(row, col, num):
14     if num in board[row]:
15         return False
16
17     if num in [board[r][col] for r in range(9)]:
18         return False
19
20     box_r, box_c = (row // 3) * 3, (col // 3) * 3
21     for r in range(box_r, box_r + 3):
22         for c in range(box_c, box_c + 3):
23             if board[r][c] == num:
24                 return False
25
26     return True

```

```

28 def sdk():
29     for r in range(9):
30         for c in range(9):
31             if board[r][c] == 0:
32                 for num in range(1, 10):
33                     if is_safe(r, c, num):
34                         board[r][c] = num
35                         if sdk():
36                             return True
37                         board[r][c] = 0
38         return False
39     return True

```

7		2		4	8			
	8	5				7		
					2	6		3
			6		1	4		2
1		8	2					7
9			7			8		
					3		4	1
8	3			9			7	
	9		4	2			3	

Q&A
